

1. COMMAND STRUCTURE

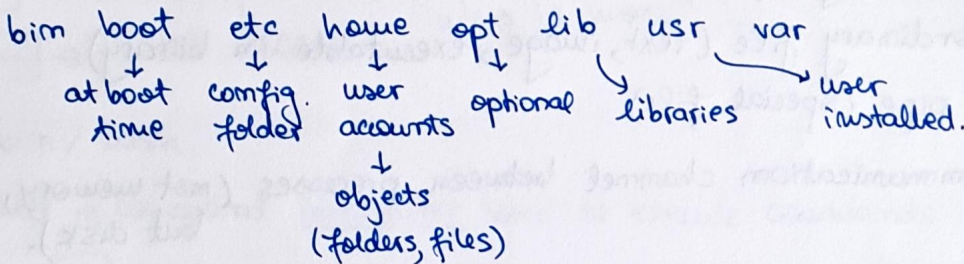
"space" = separator

folders = containers for other things (files/folders).

`ls` → List Structure → content of current folder.

`ls /etc`
↳ "abs.path"

Root: `/`
of the
file system



`ls /home` → only the first level in the folder home.

`ls` ⇔ `ls .`

2. ABSOLUTE VS RELATIVE PATH.

↓
/.../.../
↓
root

↓
../ go to the parent folder
./ current folder (HERE).

3. COMMANDS.

`ls -l` → Long list format (details about objects)

(=)

`ls --ll`

`ls --all` → see hidden files as well
(.,..)

Putty $\rightarrow$ ssh $\rightarrow$ remote to a $\neq$ computer
(terminal)

$\hookrightarrow$ secure shell $\Rightarrow$ encrypted commands / connection

/home/examiner/horea > ls

? $\sim$ $\downarrow$
group owner
owner

> ls -a // all files, even hidden ones.

> cat .bash-profile

> ls /etc $\rightarrow$ all files from that location

> ls -l /etc $\rightarrow$ long list format

first character on the line: object type.

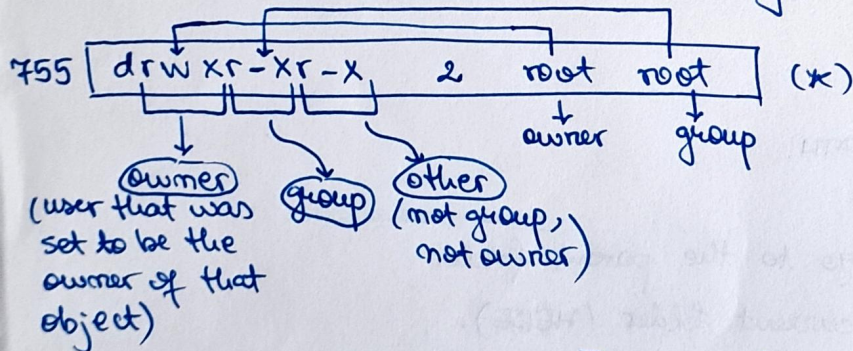
[d] $\rightarrow$ directory (folder)

[-] $\rightarrow$ ordinary file (text, image, executable in binary)

[p] $\rightarrow$ pipe (special file)

Pipe = communication channel between processes (not memory, but disk).
= "I" ?

[l] $\rightarrow$ link between disks (see storage)



r w x
 $\downarrow$
read permission given

= w x
 $\downarrow$
read permission not given

$r=4$
 $w=2$
 $x=1$
 $- = 0$ $\rightarrow$ compute numerical representation of these 3 groups of permissions.

EX: ~~4+2+1+0~~ For (*):

$rwX \rightarrow 4+2+1=7$
 $r-X \rightarrow 4+0+1=5$
 $r-X \rightarrow 4+0+1=5$ $\Rightarrow 755$.

> man ls

> man open

> man 2 open

> man 3 open

> man 2 chown → change file owner and group, used to change permission.

> ls -al ./

> less → paginate the content of output (scroll up + down)

> less /etc/passwd → users defined on the system (all)
first time: user.

! SEE ALSO at the end of man →

other manual pages, other commands...

[q] → quit manual.

[.] → current folder

[..] → parent folder

"less is more" :-

/bin / bash

Shell → command interpreter used to execute commands.

I. [sh]

ssh → secure sh.

II. [bash] (newer)

improved sh

III. [csh]

IV. [ksh]

INTERPRETERS.

> ls -l /etc/passwd

> ls -l /home/xm/

> ls -l /home/xm/exam/os/practice | less

output

input

"PIPE"

[ls] List Structure

[-h]

(attached to sth)

(-lh)

(du -h)

→ 1024 (binary)

→ "human-readable"

bytes

KB, MB, GB...

[-H]

→ 1000 (decimal) → " - " .

Optimal

we ARE inside a directory.

\$, # = prompts $\Rightarrow$ ex: maria@ubuntu: ~\$ (normal)
 $\downarrow \searrow$ root (admin)
 normal user $\downarrow$ has all permissions.
 root @ ubuntu: ~# (admin)

$\boxed{df} \rightarrow$ disk free

[du] → disk usage

↓ less } paginated output (→ less is better)
↓ more }

↓ more

{ **head** → display specif. nr. of lines from the beginning of output
ex: head -3 filename ⇒ first 3 lines.

tail → display " — " from the end of output.

cat → entire content of file once. (display)

[cut] → tokenize

□ $\rightarrow$ "what" are doing the users

who → which users are connected

last → connections during the last month.

history → last 1000 ~~connections~~. commands.

chmod → change permissions.

chown → change owner / group.

> tail -2 /etc/passwd => tudor
jamis

> tail -2 /etc/passwd | head -1 => tudor
 output input
 (only the first line)

(no filename in the syntax here) bc we have pipes.

uptime → time since server started.

date → current date

sort → lexicographic sort.

uniq → after sorting, choose only one line from each group...

grep → case sensitive → searching for occurrences...

egrep → searching through files using previous expression.

grep -i → case insensitive

grep -v → does not contain.

→ replace ...
→ delete ...

→ substitution (usually).

sed → edit input stream as a stream (flow of characters).

awk → pattern-matching program (constructions to make computations).

↳ BEGIN, END.

↳ process text on columns / fields. (tables!)

ex: sed s ?

vi / vim → complex

emacs → more than a text editor

joe → "simplest"

nano

= pico

TEXT EDITORS.

cut -d : -f 1,2,3 /etc/passwd

↓
delimiter
(space)

display 1st, 2nd, 3rd token.

ps → processes (current ones) → aux
→ efx
→ fax

ping
netstat } networking

touch → zero-size file

`cd` → change directory.

`mkdir` → make directory

`cp` → copy files

`mv` → move, rename files/folders

`rm` → remove (files)

`rmdir` → remove (folders)

`echo` → display/writes into/prints...

`file` → info abt. the object given as parameter.

a.c → file a.c. → "c program file"

hey.txt → file hey.txt → "empty" / "Ascii text".

`find` → find/search for files/folders...

`locate` → uses index (faster than find), but that index needs to be built every time.

? Regular expressions

(*) character $\begin{cases} \text{meaning}_1 \\ \text{meaning}_2 \end{cases}$ → escape original meaning.

`[.]` → matches any single character

`[\]` → escape (changes the meaning of the character following it)

`[abc]` → matches any single character that appears in the list (a, b or c)

`[a-z]` → "—" that belongs to the range (any lower-case letter).

`[^0-9]` → "—" that does not belong to the range (anything not a digit)
↓
negation.

? `[^]` → beginning of line } specification

`[$]` → end of line

`[<]` → beginning of word.

`[>]` → end of word.

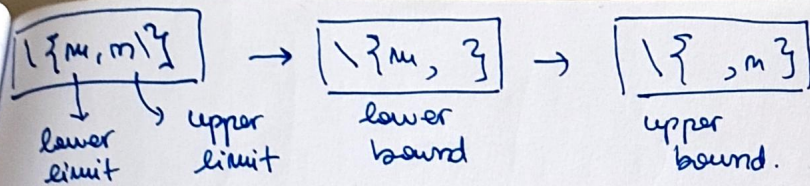
`[(\)]` → group ~~expression~~ characters into expression.

`[*]` → previous expression zero or more times.

`[+]` → previous expression one or more times.

`[?]` → previous expression zero or one times.

group " " $\rightarrow$ `[ ]`



$\boxed{\|}$ → logical OR between parts of the regular expression.

1) $.^*$ - any sequence of characters.

2) $[a-zA-Z0-9]$ - any letter, ^{any} even digit.

3) $[]$ - space

4) $^{\wedge} \underbrace{[0-9]}_{\text{at least once}} \backslash + \$$ - non-empty lines containing any characters except dig

? 5) $\backslash ([Nm] [0o] \backslash) \backslash +$ - any refusal

$\boxed{\|}$ → evaluated as expression, not command.

$\boxed{" "}$ → evaluated as variable, command.